Problema de la mochila (knapsack problem)

Dominguez Solis Roman

Instituto Politécnico Nacional
Escuela Superior de Física y Matemáticas

Programación II

7 de junio de 2022

Contenido

1 Introducción
- Historia
- Definicion

2 Simplificaciones y generalizaciones
- Problema de la mochila
- Problema de la mochila de multiple eleccion
- El problema de la suma de subconjuntos

3 Metodos de resolucion
- Algoritmos voraces
- Algoritmos genetico
- Algoritmos con programación dinámica

4 Problema de la mochila
- Complejidad y Eficiencia del programa.
- Ejemplo del algoritmo

5 Referencias

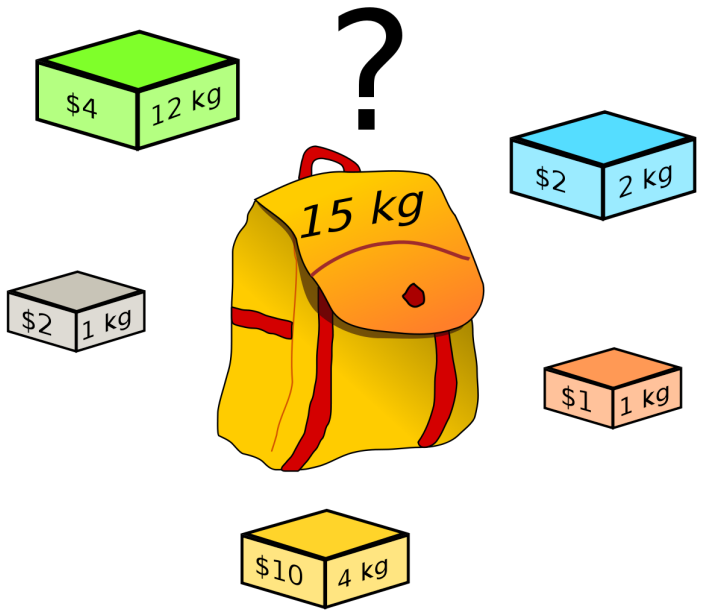
ESFM
Escuela Superior de Física y Matemáticas
Instituto Politécnico Nacional

Introducción

Escuela Superior de Física y Matemáticas

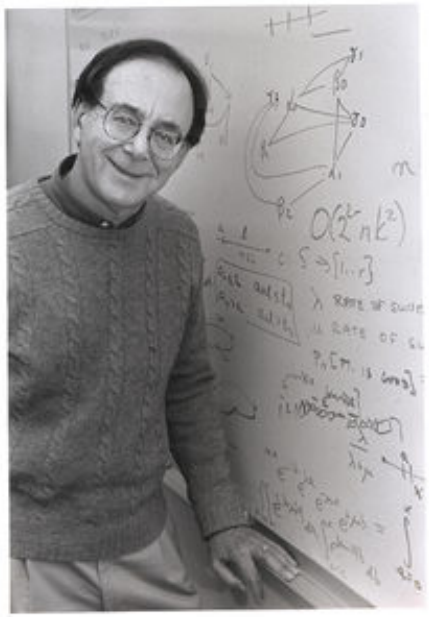
Introducción

El problema de la mochila (Knaspack problem) es un problema clásico en los problemas denominados COP (por sus siglas en inglés Combinatorial Optimization Problem – Problemas de Optimización Combinatoria) de Inteligencia Artificial. Este problema es considerado NP (Non Probabilistic Problem) ya que existe una combinación exponencial de instancias que, en su totalidad, no pueden ser resueltas.



Historia

El problema de la mochila es uno de los 21 problemas NP-completos de Richard Karp, establecidos por el informático teórico en un famoso artículo de 1972.1 Ha sido intensamente estudiado desde mediados del siglo XX y se hace referencia a él en el año 1897, en un artículo de George Mathews Ballard.



Definicion

El problema de la mochila (KP) puede ser definido con un conjunto de n artículos donde cada artículo es identificado por nx, con un valor entero px, y un peso wx. El problema consiste en elegir un subconjunto de n artículos maximizando el beneficio obtenido considerando el peso total de los artículos seleccionados, sin exceder la capacidad c de la mochila.

ESFM
Escuela Superior de Física y Matemáticas
Instituto Politécnico Nacional

Simplificaciones y generalizaciones

Problema de la mochila

El problema consiste en meter en la mochila los ítems de tal forma que se maximice el valor de los ítems que contiene, siempre que no se supere el peso máximo que puede soportar la misma. La solución al problema vendrá dada por la secuencia de variables x1, x2, ..., xn donde el valor de xi indica cuantas copias se meterán en la mochila del tipo de ítem i.

$$maximize \sum_{j=1}^{n} p_j x_j \tag{1}$$

$$subject \quad to \sum_{j=1}^{n} w_j x_j \leq c, \tag{2}$$

$$x_j \in \{0,1\}, j = 1, \ldots, n. \tag{3}$$

Figura: Formulación matematica del problema de la mochila

Problema de la mochila de multiple eleccion

Si en un problema de la mochila 0-1 los ítems están subdivididos en k clases, denotadas por N_i , y exactamente un ítem tienen que ser tomado de cada clase, entonces hablamos del problema de la mochila de múltiple elección.Se tiene como función objetivo, maximizar el beneficio

$$maximize \sum_{j=1}^{n} w_j x_j \tag{1.4}$$

$$subject \quad to \sum_{i=1}^{k} w_j x_j \leq c, \tag{2.4}$$

$$x_j \in \{0,1\}, j = 1,\ldots,n. \tag{3.4}$$

Figura: Problema de la mochila 0-1

El problema de la suma de subconjuntos

Existe una variante en la cual el beneficio es igual al peso para cada uno de los elementos. Esto no implica que el beneficio (o peso) sea el mismo para cada uno de los elementos.

$$\sum_{i=1}^{n} w_i x_i \leq c$$

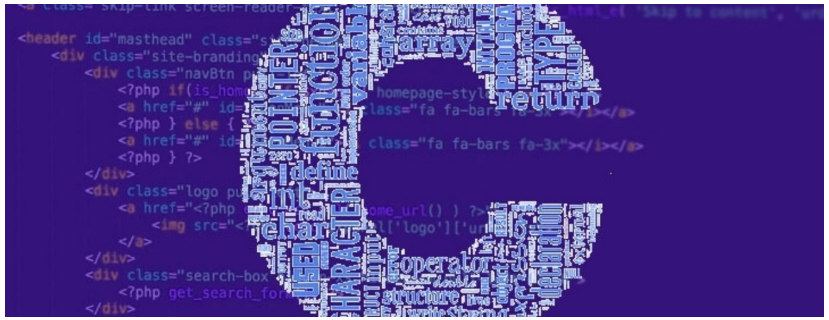
$$max \rightarrow z = \sum_{i=1}^{n} w_i x_i$$

Figura: Problema de la mochila suma de conjuntos

Metodos de resolucion

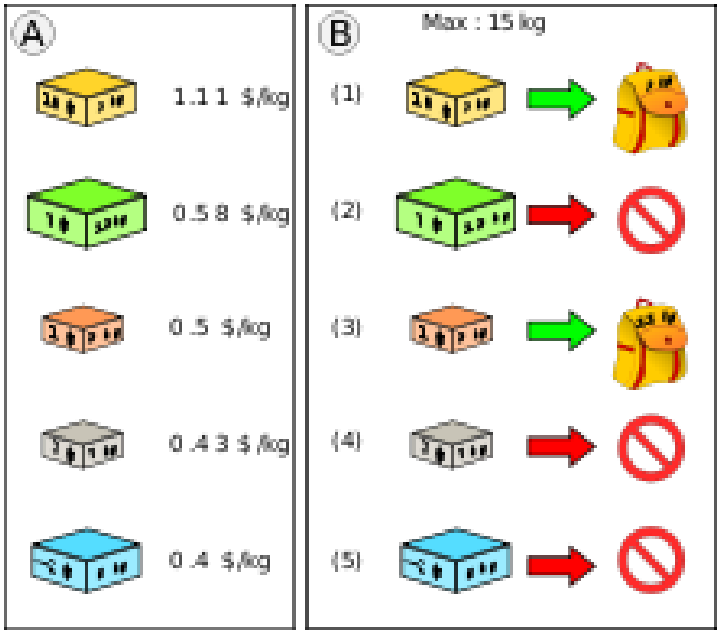
Metodos de resolucion

Este problema se ha resuelto tradicionalmente mediante programación lineal entera. El hecho de que se trate de programación lineal hace referencia a que la funciona optimizar y las inecuaciones que constituyen las restricciones han de ser lineales, es decir, han de ser funciones cuyas incógnitas están elevadas exclusivamente a la unidad.



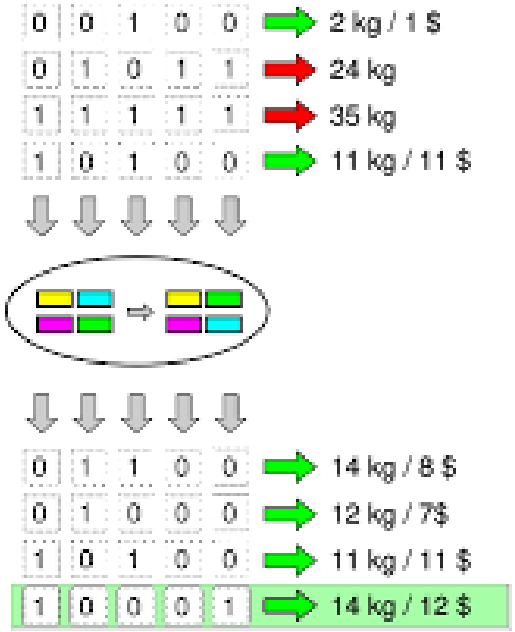
Algoritmos voraces

La idea es añadir un objeto prioritario mas efectivo hasta la saturación de la mochila. La complejidad de estos algoritmos se basa en la inversa de la calidad esperada. Podemos ver las dos fases del algoritmo voraz.
a) El ordenamiento de las cajas según un interes (en este caso en peso en valor/kg)
b) Si es posible la entrar a la mochila.



Algoritmos genetico

Se utiliza a menudo en problemas de optimización difíciles, como este, facil de implementar y obtener rapidamente una solución.



Los objetos son utilizados para el algoritmo voraz, por ejemplo el genoma (0,1,0,0,0) corresponde a un cuadro de selección de 12 kg de peso 7.

Algoritmo con Programación dinámica

El problema de la mochila tiene la propiedad que le permite utilizar el metodo de resolución de programación dinámica. Este algoritmo tiene una complejidad en tiempo y espacio que tiene como ventajas la velocidad y no hay necesidad de ordenar las variables, pero como desventajas es que gasta mucha memoria, por lo tanto no puede ser solución para problemas grandes.

Artículo	Volumen (l)	Beneficio (€)	0	1	2	3	4	5	6	7	8
estado inicial			0	0	0	0	0	0	0	0	0
A	1	2	0	2 A	2 A	2 A	2 A	2 A	2 A	2 A	2 A
B	2	5	0	2 A	5 B	7 A+B	7 A+B	7 A+B	7 A+B	7 A+B	7 A+B
C	4	6	0	2 A	5 B	7 A+B	7 A+B	8 A+C	11 B+C	13 A+B+C	13 A+B+C
D	5	10	0	2 A	5 B	7 A+B	7 A+B	10 D	12 A+D	15 B+D	17 A+B+D
E	7	13	0	2 A	5 B	7 A+B	7 A+B	10 D	12 A+D	15 B+D	17 A+B+D
F	8	16	0	2 A	5 B	7 A+B	7 A+B	10 D	12 A+D	15 B+D	17 A+B+D

¿Como funciona el algoritmo?

- Buscar una relación recurrente que conecte los valores óptimos de los diferentes problemas de la familia.
- Etapa n.

Para todo E $(0 \leq E \leq b)$ calcular $F_n^*(E)$ - $F_n^*(E) = 0$ si $E > b - a_n$

- $F_n^*(E) = c_n$ si $E \leq b - a_n$ y $c_n \geq 0$

- $F_n^*(E) = 0$ si $E \leq b - a_n$ y $c_n < 0$

- Etapa n-1

$$F_{n-1}^*(E) = Max\{c_{n-1} + F_n^*(E + a_{n-1}), F_n^*(E)\}$$

- Etapa i

$$F_i^*(E) = Max\{c_i + F_{i+1}^*(E + a_i), F_{i+1}^*(E)\}$$

Escuela Superior de Física y Matemáticas

¿Como funciona el algoritmo?

Finalmente obtenemos la Obtención de la solución óptima.

- Para i = 1, ..., n 1

$$x_i = \begin{cases} 0 & \text{si } F_i^*(E) = F_{i+1}^*(E) \\ 1 & \text{si } F_i^*(E) = c_i + F_{i+1}^*(E + a_i) \end{cases}$$

- Finalmente

$$x_n = \begin{cases} 0 & \text{si } F_n^*(E) = 0 \\ 1 & \text{si } F_n^*(E) = c_n \end{cases}$$

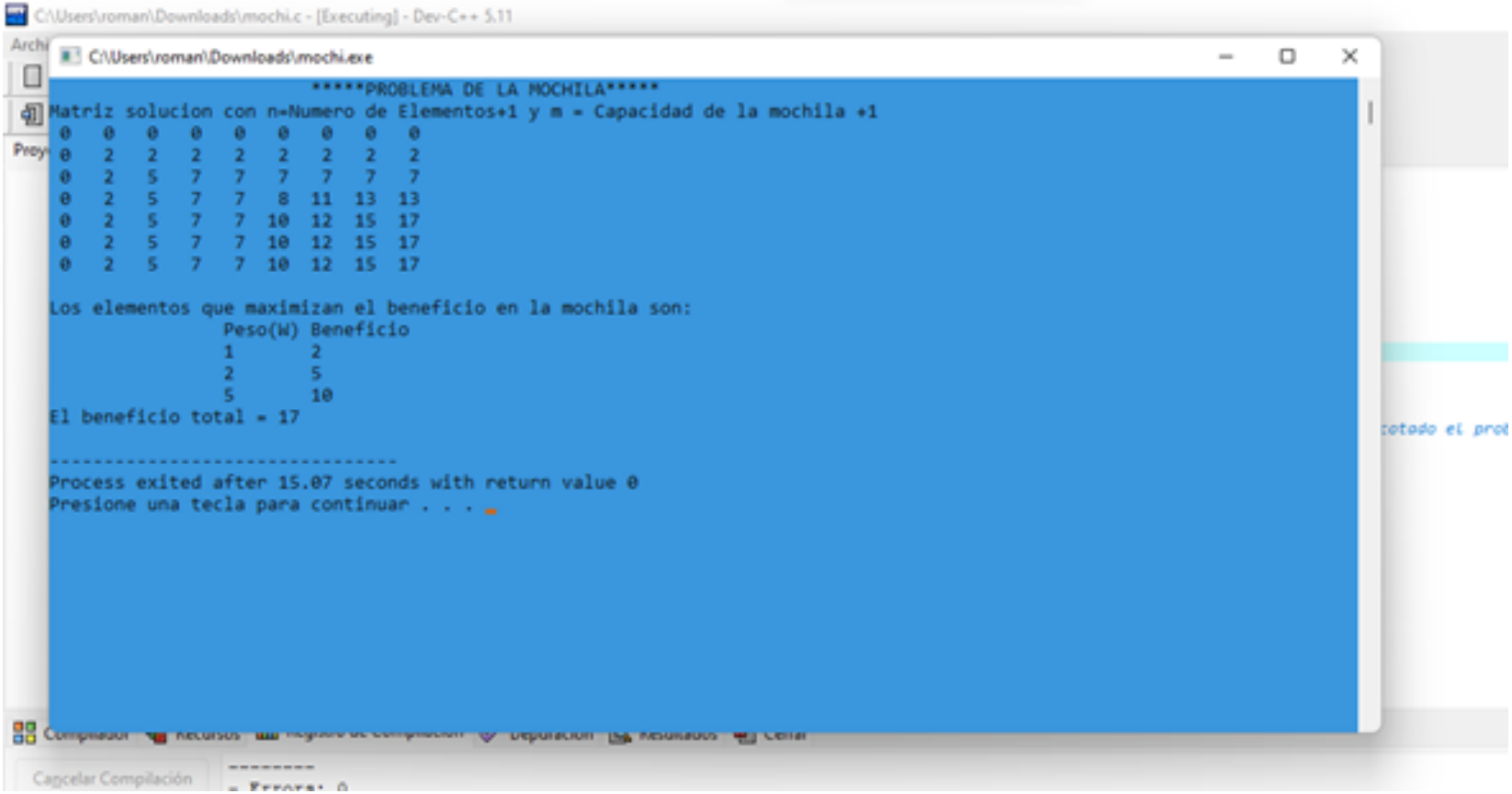
Problema de la mochila

Complejidad y eficiencia del programa.

Para los problemas de mochila unica (un solo contenedor) existen algoritmos cuya complejidad temporal y espacial es pseudo polinomial, es decir, están acotados por un polinomio en función de n y c, por ejemplo O(nc)

ESFM
Escuela Superior de Física y Matemáticas
Instituto Politécnico Nacional

Ejemplo del algoritmo



Referencias

Escuela Superior de Física y Matemáticas

Referencias

- *Hih, W., (1979). A branch and bound method for the multiconstraint zero-one knapsack problem, Journal of Operation Research Society, 369-378..*
- *Meida, F., Gim´enez, D., y L´opez J.J. (2010). Comparaci´on de metaheur´ısticas para la obtenci´on de modelos de ecuaciones simult´aneas. Congreso Espa~nol deMetaheur´ısticas, Algoritmos Evolutivos y Bionispirados 2010.*
- *Isinger, D. (2003). Where are the hard knapsack problems?. Technical Report 2003/8, DIKU, University of Copenhagen, Denmark.*